Curso/Turma: ENFGENHARIA DE SOFTWARE/ 1 PERIODO Disciplina: ALGORÍTMO e ESTRUTURA DE DADOS I Tipo: P2 Turmas: ADELLE e SNYDER	Professor(a): HILSON SILVA Data: / /2026	**TRABALHO AVALIATIVO:** **Data de Entrega no dia da aplicação da P2**

Torne-se o líder que você quer ser!

TRABALHO PRÁTICO

Estrutura de Dados

Implementação de Estruturas Lineares, Não-Lineares e Algoritmos de Ordenação Aplicados a Problemas Reais do Cotidiano

2026-2

PREENCHIMENTO DA FICHA(OBRIGATÓRIO)

Disciplina: Estrutura de Dados I

Linguagens permitidas: C, Python ou Java

NOME DA TURMA: _____

PERIODO : _____

Dupla:

ALUNO 1: _____

ALUNO 2: _____

Matrícula 1: _____

Matrícula 2: _____

Sumário

1. Introdução e Objetivos 5
1.1 Objetivos Específicos 5
1.2 Linguagem de Programação 5
1.3 Requisitos Gerais de Entrega 5
2. Estruturas de Dados Lineares 6
2.1 Filas (Queues) 6
Justificativa de Escolha 6
Situação Problema 1 – Sistema de Atendimento de uma Clínica Médica 6
Situação Problema 2 – Fila de Impressão em Laboratório de Informática 6
2.2 Pilhas (Stacks) 7
Justificativa de Escolha 7
Situação Problema 3 – Sistema de Desfazer (Undo) em Editor de Texto Simples 7
Situação Problema 4 – Verificação de Expressões com Parênteses, Colchetes e Chaves 7
2.3 Listas (Listas Encadeadas / Listas Dinâmicas) 7
Justificativa de Escolha 7
Situação Problema 5– Gerenciamento de Playlist de Músicas 8
Situação Problema 6 – Controle de Estoque de uma Pequena Loja 8
3. Estruturas de Dados Não-Lineares 8
3.1 Árvore Binária de Busca (BST) 8
Justificativa de Escolha 8
Situação Problema 7 – Cadastro e Busca de Alunos por Matrícula 8
Situação Problema 8 – Sistema de Autocomplete de Nomes de Produtos 9
3.2 Árvore Balanceada (AVL ou Rubro-Negra) 9
Justificativa de Escolha 9
Situação Problema 9 – Índice de CPF para Sistema de RH 9
Situação Problema 10 – Ranking de Jogadores em um Jogo Online 9
3.3 Grafos 10
Justificativa de Escolha 10
Situação Problema 11 – Roteamento de Entrega de Mercadorias (Caminho Mínimo) 10
Situação Problema 12 – Rede de Amizades em uma Rede Social Simples 10
3.4 Tabela Hash (Hash Table) 10
Justificativa de Escolha 10
Situação Problema 13 – Cadastro de Usuários com Login Rápido 10
Situação Problema 14 – Contagem de Frequência de Palavras em um Texto 11
4. Algoritmos de Ordenação 11
4.1 Bubble Sort 11
Justificativa de Escolha 11
Situação Problema 15 – Ordenação de Notas de uma Turma Pequena 11

Situação Problema 16 – Ordenação de Produtos por Preço em uma Vitrine Pequena 11
4.2 Selection Sort 12
Justificativa de Escolha 12
Situação Problema 17 – Seleção dos Melhores Candidatos em um Processo Seletivo 12
Situação Problema 18 – Organização de Tarefas por Prioridade (Valor Numérico) 12
4.3 Insertion Sort 12
Justificativa de Escolha 12
Situação Problema 19 – Inserção de Novas Cartas em um Jogo de Cartas Ordenado 12
Situação Problema 20 – Atualização de Ranking de Vendedores do Mês 13
4.4 Merge Sort 13
Justificativa de Escolha 13
Situação Problema 21 – Ordenação de Histórico de Transações Bancárias 13
Situação Problema 22 – Mesclagem de Duas Listas de Contatos Ordenadas 13
4.5 Quick Sort 14
Justificativa de Escolha 14
Situação Problema 23 – Ordenação de Catálogo de Produtos de E-commerce 14
Situação Problema 24 – Classificação de Resultados de uma Competição Esportiva 14
5. Orientações de Desenvolvimento e Avaliação 14
5.1 Organização do Código 14
5.2 Relatório Técnico 15
5.3 Critérios de Avaliação (Sugestão) 15
5.4 Observações Importantes 15
6. Sugestão de cronograma para o Desenvolvimento do projeto 15
7. Considerações Finais 16

1. Introdução e Objetivos

Este trabalho prático tem como objetivo principal a aplicação de conceitos fundamentais de Estrutura de Dados na resolução de problemas reais do cotidiano. A dupla deverá implementar soluções utilizando uma das três linguagens permitidas (C, Python ou Java), demonstrando domínio teórico e prático das estruturas lineares, não-lineares e dos principais algoritmos de ordenação.

A escolha de problemas reais visa aproximar o conteúdo acadêmico de situações cotidianas, permitindo que o estudante compreenda a importância e a aplicabilidade de cada estrutura de dados em cenários concretos.

1.1 Objetivos Específicos

1. Implementar estruturas de dados lineares (Filas, Pilhas e Listas) aplicadas a dois problemas reais para cada tipo;
2. Implementar estruturas de dados não-lineares (Árvore Binária, Árvore Balanceada, Grafos e Tabela Hash) aplicadas a dois problemas reais para cada tipo;
3. Implementar os algoritmos de ordenação (Bubble Sort, Selection Sort, Insertion Sort, Merge Sort e Quick Sort) aplicados a dois problemas reais para cada algoritmo;
4. Justificar, de forma clara e fundamentada, a escolha de cada estrutura de dados e algoritmo para o problema proposto;
5. Apresentar código funcional, documentado e acompanhado de testes com dados reais ou realistas.

1.2 Linguagem de Programação

A dupla poderá utilizar C, Python ou Java. Recomenda-se Python pela facilidade de prototipagem e legibilidade, especialmente em trabalhos acadêmicos com foco em estruturas de dados. Independente da linguagem escolhida, o código deve ser claro, bem comentado e organizado em módulos ou classes quando apropriado.

1.3 Requisitos Gerais de Entrega

1. **Código-fonte completo e funcional de todas as implementações;**
2. **Relatório técnico contendo a justificativa de escolha de cada estrutura/algoritmo;**
3. **Exemplos de execução com dados de entrada e saída;**
4. **Análise de complexidade de tempo e espaço (quando aplicável);**
5. **Documentação mínima (README ou comentários no código).**

2. Estruturas de Dados Lineares

As estruturas lineares organizam os dados de forma sequencial. Cada elemento possui, no máximo, um predecessor e um sucessor. São fundamentais para o gerenciamento de fluxos de dados, histórico de operações e armazenamento ordenado.

2.1 Filas (Queues)

A fila é uma estrutura do tipo FIFO (First-In, First-Out). O primeiro elemento inserido é o primeiro a ser removido. É ideal para situações em que a ordem de chegada deve ser respeitada.

Justificativa de Escolha

A fila é a estrutura mais adequada quando se deseja processar elementos na mesma ordem em que chegaram, garantindo justiça e previsibilidade no atendimento. Sua complexidade de inserção e remoção (nas extremidades) é O(1) em implementações eficientes.

Situação Problema 1 – Sistema de Atendimento de uma Clínica Médica

Contexto:

Uma clínica de atendimento ambulatorial precisa gerenciar a fila de pacientes que chegam para consulta. Os pacientes são atendidos na ordem de chegada. O sistema deve permitir a entrada de novos pacientes, a chamada do próximo paciente e a consulta do tamanho atual da fila.

O que implementar:

- Classe/estrutura Fila com operações: enqueue (chegada), dequeue (chamada), front (próximo), isEmpty e size.
- Simulação de um dia de atendimento com pelo menos 15 pacientes (imprimir nome + horário de chegada, tipo atendimento).
- Relatório final: ordem de atendimento e tempo médio de espera (estimativa).

Dados de exemplo:

Pacientes: Ana (08:05-Consulta), Bruno (08:12-Exame), Carla (08:15-Cirurgia), ... (gerar lista realista).

Situação Problema 2 – Fila de Impressão em Laboratório de Informática

Contexto:

Em um laboratório de informática da universidade, vários alunos enviam trabalhos para impressão. A impressora processa os trabalhos na ordem em que foram enviados. O sistema deve gerenciar a fila de impressão, permitindo cancelamento do último trabalho enviado (opcional) e priorização apenas por ordem de chegada.

O que implementar:

- Fila de trabalhos de impressão (nome aluno, nome do arquivo, número de páginas).
- Operações: enviar trabalho, imprimir próximo, listar fila, cancelar último (usando conceito auxiliar se necessário).
- Simulação com 10 a 20 trabalhos e relatório de sequência de impressão.

2.2 Pilhas (Stacks)

A pilha é uma estrutura do tipo LIFO (Last-In, First-Out). O último elemento inserido é o primeiro a ser removido. É essencial para controle de fluxo, desfazer ações e avaliação de expressões.

Justificativa de Escolha

A pilha é a estrutura natural quando se precisa reverter a ordem de processamento ou manter o histórico das últimas ações realizadas. Operações de empilhar e desempilhar são O(1).

Situação Problema 3 – Sistema de Desfazer (Undo) em Editor de Texto Simples

Contexto:

Um editor de texto simples precisa oferecer a funcionalidade de desfazer as últimas ações do usuário (digitação, exclusão, formatação). Cada ação deve ser registrada de forma que a mais recente possa ser revertida primeiro.

O que implementar:

- Pilha de ações (tipo de ação + conteúdo/estado anterior).
- Operações: executar ação (push), desfazer (pop), refazer (opcional com segunda pilha).
- Simulação de uma sequência de 10 –15 ações e demonstração do desfazer passo a passo.

Situação Problema 4 – Verificação de Expressões com Parênteses, Colchetes e Chaves

Contexto:

Em compiladores e editores de código, é necessário verificar se expressões matemáticas ou trechos de código possuem parênteses, colchetes e chaves balanceados. A pilha é a estrutura clássica para essa verificação.

O que implementar:

- Função que recebe uma string e retorna se está balanceada ou não.
- Suporte a (), [] e {}.
- Testes com expressões válidas e inválidas (ex.: "{ [( )] }", "([ )]", "((()))", "([)]").
- Relatório de erros indicando a posição do primeiro desbalanceamento (opcional, mas valorizado).

2.3 Listas (Listas Encadeadas / Listas Dinâmicas)

As listas permitem armazenamento sequencial com inserção e remoção em qualquer posição. Podem ser simplesmente ou duplamente encadeadas. São flexíveis quando o tamanho dos dados varia dinamicamente.

Justificativa de Escolha

Listas são escolhidas quando é necessário inserir, remover ou reordenar elementos em posições intermediárias com frequência, e quando o tamanho máximo dos dados não é conhecido a priori. Oferecem flexibilidade superior a arrays estáticos.

Situação Problema 5– Gerenciamento de Playlist de Músicas

Contexto:

Um aplicativo de música permite ao usuário criar e gerenciar playlists. É possível adicionar músicas no início, no final ou em qualquer posição, remover músicas e reordenar a lista.

O que implementar:

- Lista encadeada de músicas (título, artista, duração).
- Operações: inserir no início, no final, em posição específica, remover por título, listar, buscar.
- Simulação de criação e edição de uma playlist com pelo menos 12 músicas.

Situação Problema 6 – Controle de Estoque de uma Pequena Loja

Contexto:

Uma loja de materiais de construção precisa manter o controle de produtos em estoque. Novos produtos podem ser cadastrados, quantidades atualizadas e produtos esgotados removidos da lista ativa.

O que implementar:

- Lista de produtos (código, nome, quantidade, preço unitário).
- Operações: cadastrar, atualizar quantidade, remover produto, buscar por nome/código, listar produtos abaixo de estoque mínimo.
- Simulação com entrada e saída de mercadorias ao longo de uma semana.

3. Estruturas de Dados Não-Lineares

As estruturas não-lineares organizam dados de forma hierárquica ou em rede, permitindo relações mais complexas entre os elementos. São essenciais para busca eficiente, modelagem de relacionamentos e indexação.

3.1 Árvore Binária de Busca (BST)

Uma árvore binária de busca mantém os elementos ordenados: valores menores à esquerda e maiores à direita. Permite buscas, inserções e remoções com complexidade média O(log n).

Justificativa de Escolha

A BST é escolhida quando se precisa de busca, inserção e remoção frequentes mantendo os dados ordenados, sem a necessidade de reordenar toda a coleção a cada operação. É ideal para dicionários e índices simples.

Situação Problema 7 – Cadastro e Busca de Alunos por Matrícula

Contexto:

A secretaria acadêmica precisa cadastrar alunos e realizar buscas rápidas por número de matrícula. A árvore binária de busca permite manter os alunos ordenados e localizar qualquer matrícula de forma eficiente.

O que implementar:

- Árvore BST com nós contendo matrícula, nome e curso.
- Operações: inserir, buscar por matrícula, remover, percurso em ordem (listagem ordenada).

- Cadastro de pelo menos 20 alunos e demonstração de buscas e listagem ordenada.

Situação Problema 8 – Sistema de Autocomplete de Nomes de Produtos

Contexto:

Um e-commerce simples deseja sugerir produtos enquanto o usuário digita. Uma árvore binária (ou trie simplificada via BST de prefixos) pode ser usada para armazenar nomes de produtos e retornar sugestões ordenadas.

O que implementar:

- BST de nomes de produtos.
- Função que, dado um prefixo, retorna todos os produtos que começam com aquele prefixo (percurso limitado).
- Base de pelo menos 30 produtos e testes de autocomplete.

3.2 Árvore Balanceada (AVL ou Rubro-Negra)

Árvores balanceadas garantem que a altura permaneça O(log n) mesmo no pior caso, através de rotações após inserções e remoções. A AVL é a mais didática para trabalhos acadêmicos.

Justificativa de Escolha

Quando a performance de busca no pior caso é crítica (ex.: sistemas com milhares de registros e necessidade de resposta rápida), a árvore balanceada evita a degeneração da BST comum em lista encadeada.

Situação Problema 9 – Índice de CPF para Sistema de RH

Contexto:

O setor de Recursos Humanos de uma empresa precisa de um índice rápido de funcionários por CPF. A árvore AVL garante que mesmo com milhares de funcionários a busca permaneça eficiente.

O que implementar:

- Árvore AVL com CPF (chave), nome e cargo.
- Inserção com balanceamento, busca, remoção e verificação de altura/fator de balanceamento.
- Inserção de pelo menos 25 funcionários e demonstração de que a árvore permanece balanceada.

Situação Problema 10 – Ranking de Jogadores em um Jogo Online

Contexto:

Um jogo online mantém um ranking de jogadores por pontuação. A árvore balanceada permite inserir novas pontuações e consultar rapidamente os melhores jogadores, mantendo a estrutura eficiente mesmo com muitas atualizações.

O que implementar:

- Árvore AVL ordenada por pontuação (ou por pontuação + nome como desempate).
- Operações de inserção/atualização de pontuação e consulta dos Top-K jogadores.
- Simulação de 30 jogadores com atualizações de pontuação.

3.3 Grafos

Grafos modelam relações entre entidades (vértices ligados por arestas). Podem ser direcionados ou não, ponderados ou não. São fundamentais para redes, rotas e dependências.

Justificativa de Escolha

Grafos são a estrutura ideal quando o problema envolve conexões, caminhos, redes de relacionamento ou dependências entre elementos. Algoritmos como BFS, DFS e Dijkstra resolvem problemas clássicos de caminho mínimo e conectividade.

Situação Problema 11 – Roteamento de Entrega de Mercadorias (Caminho Mínimo)

Contexto:

Uma empresa de delivery precisa calcular a rota de menor custo (ou menor tempo) entre o centro de distribuição e os pontos de entrega. O mapa da cidade é modelado como um grafo ponderado.

O que implementar:

- Grafo ponderado (lista de adjacência) representando bairros/pontos de entrega.
- Algoritmo de Dijkstra (ou BFS se não ponderado) para encontrar o caminho de menor custo.
- Pelo menos 10 vértices e demonstração de rotas entre diferentes pares de pontos.

Situação Problema 12 – Rede de Amizades em uma Rede Social Simples

Contexto:

Uma rede social simplificada deseja descobrir amigos em comum, sugerir conexões e verificar se duas pessoas estão conectadas através de amigos (graus de separação).

O que implementar:

- Grafo não-direcionado de usuários.
- Operações: adicionar amizade, listar amigos, amigos em comum, BFS para graus de separação.
- Rede com pelo menos 15 usuários e consultas de relacionamentos.

3.4 Tabela Hash (Hash Table)

A tabela hash oferece acesso médio O(1) a partir de uma chave. Utiliza uma função de espalhamento e tratamento de colisões (encadeamento ou endereçamento aberto).

Justificativa de Escolha

Quando a operação predominante é busca, inserção e remoção por chave e a ordem dos elementos não importa, a tabela hash é a estrutura mais eficiente em tempo médio. Ideal para dicionários, caches e índices.

Situação Problema 13 – Cadastro de Usuários com Login Rápido

Contexto:

Um sistema web precisa autenticar usuários rapidamente a partir do e-mail ou nome de usuário. A tabela hash permite localizar a senha (hash) do usuário em tempo constante médio.

O que implementar:

- Tabela hash com tratamento de colisões (encadeamento recomendado).

- Operações: cadastrar usuário, autenticar (login), remover conta, listar (opcional).
- Pelo menos 20 usuários e testes de colisão e autenticação.

Situação Problema 14 – Contagem de Frequência de Palavras em um Texto

Contexto:

Um analisador de textos precisa contar quantas vezes cada palavra aparece em um documento (ex.: análise de sentimentos, nuvem de palavras, detecção de plágio simples).

O que implementar:

- Tabela hash onde a chave é a palavra e o valor é a contagem.
- Processamento de um texto real (notícia, artigo ou livro curto).
- Exibição das palavras mais frequentes e total de palavras distintas.

4. Algoritmos de Ordenação

Os algoritmos de ordenação organizam dados segundo um critério (crescente, decrescente, por múltiplos campos). Embora o enunciado mencione "Algoritmos de Buscas", os algoritmos listados (Bubble, Selection, Insertion, Merge e Quick) são clássicos de ordenação e serão tratados como tal. A ordenação é pré-requisito para buscas binárias eficientes e para apresentação ordenada de resultados.

4.1 Bubble Sort

O Bubble Sort compara elementos adjacentes e os troca se estiverem fora de ordem, fazendo os maiores "subirem" como bolhas. Complexidade O(n²). Didático e simples, porém ineficiente para grandes volumes.

Justificativa de Escolha

Escolhido principalmente para fins didáticos e para conjuntos muito pequenos de dados, ou quando a estabilidade e a simplicidade de implementação são mais importantes que a performance.

Situação Problema 15 – Ordenação de Notas de uma Turma Pequena

Contexto:

Um professor de uma turma com poucos alunos (até 30) deseja ordenar as notas da prova para facilitar a visualização e o cálculo de estatísticas.

O que implementar:

- Bubble Sort para ordenar um vetor de notas (ou registros aluno + nota).
- Versão otimizada (parada antecipada se não houver trocas).
- Comparação do número de comparações/trocas com outros algoritmos.

Situação Problema 16 – Ordenação de Produtos por Preço em uma Vitrine Pequena

Contexto:

Uma pequena loja virtual com poucos produtos deseja ordenar a vitrine por preço crescente ou decrescente. O volume é pequeno o suficiente para o Bubble Sort ser aceitável.

O que implementar:

- Ordenação de lista de produtos (nome + preço) com Bubble Sort.

- Opção de ordem crescente/decrescente.
- Demonstração com 10–20 produtos.

4.2 Selection Sort

O Selection Sort seleciona repetidamente o menor (ou maior) elemento restante e o coloca na posição correta. Também O(n²), mas realiza menos trocas que o Bubble Sort.

Justificativa de Escolha

Útil quando o custo de troca é elevado (ex.: objetos grandes) e se deseja minimizar o número de movimentações. Também didático e de implementação simples.

Situação Problema 17 – Seleção dos Melhores Candidatos em um Processo Seletivo

Contexto:

Uma empresa precisa ordenar candidatos por pontuação de uma prova para selecionar os melhores colocados. O Selection Sort pode ser usado para encontrar sucessivamente o maior restante.

O que implementar:

- Selection Sort em lista de candidatos (nome + pontuação).
- Extração dos Top-N candidatos após a ordenação.
- Teste com pelo menos 15 candidatos.

Situação Problema 18 – Organização de Tarefas por Prioridade (Valor Numérico)

Contexto:

Um gerenciador de tarefas pessoais ordena as atividades pelo nível de prioridade (1 a 5). O Selection Sort pode ser aplicado para reorganizar a lista diariamente.

O que implementar:

- Lista de tarefas com prioridade e descrição.
- Selection Sort por prioridade.
- Simulação de um dia de trabalho com 12–15 tarefas.

4.3 Insertion Sort

O Insertion Sort constrói a sequência ordenada um elemento por vez, inserindo cada novo elemento na posição correta. Eficiente para conjuntos pequenos ou quase ordenados (O(n) no melhor caso).

Justificativa de Escolha

Excelente para dados que chegam de forma incremental ou que já estão parcialmente ordenados. É estável e tem bom desempenho prático em listas pequenas.

Situação Problema 19 – Inserção de Novas Cartas em um Jogo de Cartas Ordenado

Contexto:

Em um jogo de cartas, o jogador mantém a mão ordenada por valor. Cada nova carta recebida deve ser inserida na posição correta.

O que implementar:

- Insertion Sort (ou inserção individual) para manter a mão ordenada.

- Simulação de recebimento de 10–13 cartas.
- Visualização da mão após cada inserção.

Situação Problema 20 – Atualização de Ranking de Vendedores do Mês

Contexto:

Uma loja atualiza diariamente o ranking de vendedores por valor de vendas. Como a lista do dia anterior já está ordenada, a inserção do novo total de cada vendedor pode ser feita de forma eficiente com Insertion Sort.

O que implementar:

- Insertion Sort em lista de vendedores (nome + total de vendas).
- Simulação de atualizações diárias ao longo de uma semana.
- Comparação de desempenho com lista já ordenada vs. lista aleatória.

4.4 Merge Sort

O Merge Sort divide o vetor ao meio recursivamente, ordena as metades e as intercala. Complexidade O(n log n) no pior caso. Estável e previsível.

Justificativa de Escolha

Escolhido quando se precisa de garantia de desempenho O(n log n) e estabilidade (manter a ordem relativa de elementos iguais). Ideal para grandes volumes de dados e ordenação externa.

Situação Problema 21 – Ordenação de Histórico de Transações Bancárias

Contexto:

Um sistema bancário precisa ordenar o extrato de transações por data/hora para gerar relatórios. O volume pode ser grande e a estabilidade é desejável (transações no mesmo segundo mantêm ordem de registro).

O que implementar:

- Merge Sort em lista de transações (data, valor, descrição).
- Ordenação por data/hora.
- Teste com pelo menos 50 transações (pode ser gerado proceduralmente).

Situação Problema 22 – Mesclagem de Duas Listas de Contatos Ordenadas

Contexto:

Dois departamentos de uma empresa possuem listas de contatos já ordenadas alfabeticamente. É necessário gerar uma lista única ordenada sem duplicatas.

O que implementar:

- Uso da etapa de merge do Merge Sort para combinar duas listas ordenadas.
- Remoção de duplicatas durante a mesclagem.
- Demonstração com duas listas de 20 contatos cada.

4.5 Quick Sort

O Quick Sort escolhe um pivô, particiona o vetor e ordena as partições recursivamente. Complexidade média O(n log n), pior caso O(n²). Em geral o mais rápido na prática para arrays em memória.

Justificativa de Escolha

Preferido na maioria das aplicações reais em memória principal por seu excelente desempenho médio e baixo uso de memória auxiliar. Requer atenção ao pior caso (dados já ordenados) e escolha de pivô.

Situação Problema 23 – Ordenação de Catálogo de Produtos de E-commerce

Contexto:

Uma loja virtual com milhares de produtos precisa ordenar o catálogo por preço, nome ou popularidade de forma rápida para exibição ao usuário.

O que implementar:

- Quick Sort (com escolha de pivô mediana-de-três ou randomizado, se possível).
- Ordenação de lista de produtos por diferentes critérios.
- Teste com pelo menos 100 produtos (gerados ou lidos de arquivo).
- Comparação de tempo com Merge Sort (opcional, mas valorizado).

Situação Problema 24 – Classificação de Resultados de uma Competição Esportiva

Contexto:

Após uma corrida ou campeonato, é necessário ordenar os atletas por tempo ou pontuação para gerar a classificação final e o pódio.

O que implementar:

- Quick Sort em registros de atletas (nome, tempo ou pontos).
- Geração da classificação final e identificação do pódio (Top 3).
- Teste com pelo menos 30 atletas.

5. Orientações de Desenvolvimento e Avaliação

5.1 Organização do Código

- Separar cada estrutura/algoritmo em módulos, classes ou arquivos distintos.
- Incluir comentários explicativos nas partes principais.
- Utilizar nomes de variáveis e funções significativos.
- Tratar casos de erro (lista vazia, chave inexistente, etc.).

5.2 Relatório Técnico

Além do código, a dupla deverá entregar um relatório contendo:

- Justificativa detalhada da escolha de cada estrutura e algoritmo para os problemas selecionados.
- Complexidade de tempo e espaço de cada implementação.
- Exemplos de entrada e saída.
- Dificuldades encontradas e soluções adotadas.
- Conclusão sobre a aplicabilidade das estruturas em problemas reais.

5.3 Critérios de Avaliação (Sugestão)

Sugestão de distribuição dos 4 pontos do trabalho (o professor poderá ajustar):

- Implementação correta das estruturas lineares (Filas, Pilhas, Listas) – 25%
- Implementação correta das estruturas não-lineares (Árvores, Grafos, Hash) – 30%
- Implementação correta dos algoritmos de ordenação – 20%
- Qualidade das justificativas e do relatório – 15%
- Organização, documentação e testes – 10%
- Apresentação opcional após a P2 (Bônus de um ponto)

5.4 Observações Importantes

A dupla deve implementar as estruturas do zero (não utilizar bibliotecas prontas que já implementam a estrutura, exceto para estruturas nativas básicas da linguagem quando explicitamente permitido pelo professor da disciplina). O foco é o entendimento do funcionamento interno.

É permitido (e recomendado) o uso de arrays/listas nativas da linguagem como base para a implementação das estruturas (ex.: usar array para implementar a pilha ou a tabela hash).

Problemas adicionais ou variações criativas dos problemas propostos serão valorizados, desde que mantenham o foco nas estruturas solicitadas.

6. Sugestão de cronograma para o Desenvolvimento do projeto

Sugestão de organização do tempo (ajustável conforme o prazo da disciplina):

- Semana 1–2: Estruturas Lineares (Filas, Pilhas e Listas) + justificativas.
- Semana 3–4: Árvores (BST e AVL) + justificativas.
- Semana 5: Grafos e Tabela Hash + justificativas.
- Semana 6: Algoritmos de Ordenação + comparações de desempenho.

- Semana 7: Integração, testes finais, relatório e revisão.

7. Considerações Finais

Este escopo foi elaborado de forma bem detalhada para garantir que a dupla pratique a implementação de todas as estruturas e algoritmos solicitados, sempre conectados a problemas reais do cotidiano. A ênfase na justificativa da escolha de cada modelo desenvolve o raciocínio crítico necessário para selecionar a estrutura adequada em projetos futuros, com isso fortalecer o aprendizado do conteúdo ministrado.

O trabalho prioriza a compreensão profunda dos conceitos em detrimento do uso de soluções prontas, preparando o estudante para desafios reais de programação e análise de desempenho.

Bons Estudos!

